

SOFTWARE ENGINEERING LAB
CSE412

Version Control with
Git & GitHub

A Hands-On Lab Manual

From your first commit to merging pull requests

Project repository: [github_lab](#)

Prerequisites: Git installed · GitHub account · an IDE (VS Code)

Estimated time: 90–120 minutes

Contents

1. Before You Begin

1.1 What you will learn

This manual walks you through the complete day-to-day workflow that professional developers use with Git and GitHub. By the end of the lab you will be able to:

- Configure Git and connect your machine to GitHub.
- Create a local repository, link it to the blank `github_lab` project, and push code.
- Use the core cycle: stage → commit → push → pull.
- Work with branches and resolve a merge conflict on purpose.
- Open Issues, create Pull Requests, review code, and merge — entirely on GitHub.
- Do all of the above without leaving your IDE (VS Code).

1.2 Git vs. GitHub — the one-minute distinction

Git is the version-control tool that runs on your computer. It tracks the history of your files.

GitHub is a website that hosts Git repositories in the cloud so teams can collaborate. You use Git commands locally; you use GitHub to share and review.



NOTE

Think of it like this: **Git** is the camera that takes snapshots of your project. **GitHub** is the shared photo album in the cloud where everyone on the team can see those snapshots.

1.3 Create the blank project on GitHub

Do this first — the rest of the manual assumes the empty repository already exists.

1. Sign in to **github.com** and click the + icon (top-right) → **New repository**.
2. Repository name: `github_lab`.
3. Leave it **completely blank** — do **not** add a README, .gitignore, or license. (An empty repo makes the first push cleaner to learn from.)
4. Choose **Public**, then click **Create repository**.
5. Copy the HTTPS URL shown on the next page — it looks like `https://github.com/<your-username>/github_lab.git`.



TIP

Throughout this manual, replace `<your-username>` with your actual GitHub username wherever you see it.

2. First-Time Git Configuration

You only need to do this once per machine. Git stamps every commit with your name and email, so set them before committing anything.

Terminal

```
# Tell Git who you are (used to label your commits)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Make 'main' the default branch name for new repos
git config --global init.defaultBranch main

# See everything Git currently knows about your setup
git config --list
```

Quick reference for the commands above:

Command	What it does
<code>git config --global user.name</code>	Sets the author name attached to your commits.
<code>git config --global user.email</code>	Sets the author email (use your GitHub email).
<code>git config --list</code>	Prints all current configuration values.



NOTE

The `--global` flag applies the setting to every repository on your computer. Omit it to apply a setting to just the current project.

3. Create a Local Repository and Push Your First Code

Now you'll connect a folder on your computer to the empty `github_lab` repository and push real code to it.

3.1 Make the project folder

Terminal

```
# Create a folder and move into it
mkdir github_lab
cd github_lab

# Turn this folder into a Git repository
git init
```

After `git init`, Git creates a hidden `.git` folder that stores the entire history. You won't touch it directly.

3.2 Create your first code file

Create a file named `hello.py` with the following content. (You can do this in your IDE, or with the terminal command shown after.)

hello.py

```
# hello.py – our first file in the github_lab project

def greet(name):
    """Return a friendly greeting."""
    return f"Hello, {name}! Welcome to the Git lab."

if __name__ == "__main__":
    print(greet("Student"))
```

Prefer the terminal? This writes the same file:

Terminal

```
cat > hello.py << 'EOF'
def greet(name):
    return f"Hello, {name}! Welcome to the Git lab."

if __name__ == "__main__":
    print(greet("Student"))
EOF
```

3.3 Add a .gitignore

A **.gitignore** file tells Git which files to never track — things like caches, build output, and secrets. Create one now:

.gitignore

```
# .gitignore – files Git should ignore

# Python
__pycache__/
*.pyc
.venv/
env/

# Editor / OS noise
.vscode/
.idea/
.DS_Store

# Secrets
.env
```

3.4 Stage, commit, and connect to GitHub

This is the heart of Git. Read each step carefully — you'll repeat this cycle constantly.

Terminal

```
# 1. See what has changed (untracked files show in red)
git status

# 2. Stage your files (move them to the "ready to commit" area)
```

```
git add hello.py .gitignore
# ...or stage everything at once:
git add .

# 3. Commit — save a permanent snapshot with a message
git commit -m "Initial commit: add greet function and gitignore"

# 4. Link this local repo to your GitHub repository
git remote add origin https://github.com/<your-username>/github_lab.git

# 5. Verify the link
git remote -v

# 6. Push your commit to GitHub (-u sets 'origin main' as the default)
git push -u origin main
```

**NOTE**

The three areas of Git. Files move through three places:

Working Directory → your edits live here as you type.

Staging Area → `git add` puts changes here, ready to be saved.

Repository → `git commit` writes the staged changes into permanent history.

**WATCH OUT**

The very first push uses `git push -u origin main`. The `-u` creates a tracking link, so every push afterward is just `git push`.

If GitHub asks for a password, use a **Personal Access Token** (GitHub → Settings → Developer settings → Tokens), not your account password.

Reload your repository page on GitHub — your files are now there.

3.5 Core workflow command reference

Command	What it does
<code>git init</code>	Start tracking a new project in the current folder.
<code>git status</code>	Show what is changed, staged, or untracked.
<code>git add <file></code>	Stage a specific file for the next commit.
<code>git add .</code>	Stage every changed file in the project.
<code>git commit -m "msg"</code>	Save a snapshot of staged changes with a message.
<code>git remote add origin <url></code>	Connect the local repo to a GitHub repo.
<code>git remote -v</code>	List the remote repositories you're linked to.
<code>git push</code>	Upload local commits to GitHub.
<code>git pull</code>	Download and merge commits from GitHub.
<code>git clone <url></code>	Copy an existing GitHub repo to your computer.

4. Making More Changes & Reading History

4.1 Add a second file

Let's grow the project. Create `calculator.py`:

calculator.py

```
# calculator.py – basic arithmetic helpers

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

if __name__ == "__main__":
    print("2 + 3 =", add(2, 3))
    print("10 / 2 =", divide(10, 2))
```

Now stage, commit, and push it:

Terminal

```
git add calculator.py
git commit -m "Add calculator module with four operations"
git push
```

4.2 Inspecting what changed

Before committing, it's good practice to review your edits:

Terminal

```
# Show unstaged changes (what you've edited but not added)
git diff

# Show staged changes (what will go into the next commit)
git diff --staged
```

4.3 Reading the commit history

Terminal

```
# Full history with author, date, and message
git log
```

```
# Compact one-line-per-commit view
git log --oneline

# Visual graph of all branches – great for understanding merges
git log --oneline --graph --all

# Everything about a single commit
git show <commit-hash>
```

Command	What it does
<code>git diff</code>	Changes in your working directory, not yet staged.
<code>git diff --staged</code>	Changes that are staged and ready to commit.
<code>git log</code>	Full chronological commit history.
<code>git log --oneline</code>	One condensed line per commit.
<code>git log --oneline --graph --all</code>	ASCII graph of all branches and merges.
<code>git show <hash></code>	Full details and diff of one commit.

TIP

A commit hash is the long ID like `a1b2c3d`. You only need the first 7 characters to refer to it in most commands.

5. Branching & Merging

A branch is an independent line of work. You build a feature on its own branch so the main code stays stable, then merge it back when it's ready. This is the foundation of team collaboration.

5.1 Create and switch to a feature branch

Terminal

```
# See your branches (the * marks the current one)
git branch

# Create a new branch AND switch to it in one step
git checkout -b feature/power-function

# (Modern equivalent using switch)
# git switch -c feature/power-function
```

5.2 Do work on the branch

While on `feature/power-function`, add a new function to `calculator.py`. Add this below the `divide` function:

```
calculator.py (addition)
```



```
def power(base, exponent):  
    """Return base raised to the given exponent."""  
    return base ** exponent
```

Commit the change on this branch:

Terminal

```
git add calculator.py  
git commit -m "Add power function to calculator"  
  
# Push the branch to GitHub (first push of a new branch)  
git push -u origin feature/power-function
```

5.3 Merge the branch back into main

Once the feature works, bring it into the main branch:

Terminal

```
# Switch back to main  
git checkout main  
  
# Make sure main is up to date  
git pull  
  
# Merge your feature branch into main  
git merge feature/power-function  
  
# Push the merged result  
git push  
  
# Clean up: delete the local branch you no longer need  
git branch -d feature/power-function
```



NOTE

In a real team you usually **don't** merge locally like this. Instead you open a **Pull Request** on GitHub so others can review first — that's exactly what Section 7 covers. This local merge is shown so you understand what a PR does under the hood.

5.4 Branching command reference

Command	What it does
<code>git branch</code>	List all local branches.
<code>git branch <name></code>	Create a new branch (without switching).
<code>git checkout <name></code>	Switch to an existing branch.
<code>git checkout -b <name></code>	Create a new branch and switch to it.
<code>git switch <name></code>	Modern command to switch branches.
<code>git switch -c <name></code>	Modern command to create + switch.
<code>git merge <name></code>	Merge the named branch into the current one.

Command	What it does
<code>git branch -d <name></code>	Delete a branch that has been merged.
<code>git branch -D <name></code>	Force-delete a branch (even if unmerged).

6. Resolving a Merge Conflict (On Purpose)

Conflicts happen when two branches change the same line differently. They look scary the first time, so let's create one deliberately and fix it in a controlled way.

6.1 Set up the conflict

Create two branches that edit the **same line**. First, on `main`, make sure `hello.py` ends with this greeting line:

```
hello.py (current line)
```

```
    return f"Hello, {name}! Welcome to the Git lab."
```

Now create branch A and change that line:

```
Terminal
```

```
git checkout -b branch-a
# Edit hello.py – change the greeting to:
#     return f"Hi {name}, great to see you!"
git add hello.py
git commit -m "Branch A: change greeting wording"
```

Switch back to main and create branch B that changes the same line differently:

```
Terminal
```

```
git checkout main
git checkout -b branch-b
# Edit hello.py – change the SAME line to:
#     return f"Welcome aboard, {name}!"
git add hello.py
git commit -m "Branch B: change greeting wording differently"
```

6.2 Trigger the conflict

```
Terminal
```

```
# Go to main and merge branch A – this succeeds cleanly
git checkout main
git merge branch-a

# Now merge branch B – CONFLICT, because the same line changed
git merge branch-b
```

Git stops and marks the conflict inside `hello.py`. Open the file and you'll see conflict markers:

hello.py (conflicted)

```
<<<<<< HEAD
    return f"Hi {name}, great to see you!"
=====
    return f>Welcome aboard, {name}!"
>>>>>> branch-b
```

 NOTE

Reading the markers: the part between <<<<<< HEAD and ===== is the current branch's version. The part between ===== and >>>>>> is the incoming branch's version.

6.3 Resolve it

Edit the file by hand: delete the markers and keep whatever final version you want. For example, settle on a single line:

hello.py (resolved)

```
return f>Welcome aboard, {name}! Great to see you."
```

Then finish the merge:

Terminal

```
# Stage the resolved file
git add hello.py

# Complete the merge with a commit
git commit -m "Resolve greeting conflict between branch-a and branch-b"

git push
```

 TIP

VS Code shows conflicts with colored blocks and clickable buttons: **Accept Current**, **Accept Incoming**, or **Accept Both**. For real conflicts that's far easier than editing markers by hand.

 WATCH OUT

If you ever want to bail out of a merge mid-conflict, run `git merge --abort` to return everything to how it was before you started.

7. Collaborating on GitHub

Everything so far happened locally. Now we use GitHub's real strength: working together through Issues, Pull Requests, and code review. This is how professional teams ship software.

7.1 Creating an Issue

An Issue is a tracked to-do, bug report, or feature request. It's where work begins.

1. On your `github_lab` repo page, click the **Issues** tab → **New issue**.

2. Title: `Add a modulo (remainder) operation to calculator`.
3. In the description, explain what's needed. A good Issue is specific:

Issue description

The calculator module supports add, subtract, multiply, divide, and power.

We should add a modulo function that returns the remainder of a division.

Acceptance criteria:

- Function named `modulo(a, b)`
- Returns `a % b`
- Raises `ValueError` when `b == 0`

4. On the right, optionally add **Labels** (e.g. `enhancement`), an **Assignee**, and a **Milestone**.
5. Click **Submit new issue**. Note its number — e.g. `#1`.

TIP

Issues get numbers you can reference anywhere. Typing `#1` in a commit message or PR auto-links to that Issue. Writing `Closes #1` in a PR will **automatically close** the Issue when the PR merges.

7.2 Create a branch for the Issue

Never commit straight to main on a team. Create a dedicated branch for this work:

Terminal

```
# Make sure you start from an up-to-date main
git checkout main
git pull

# Create a branch named after the work
git checkout -b feature/modulo-operation
```

Add the modulo function to `calculator.py`:

calculator.py (addition)

```
def modulo(a, b):
    """Return the remainder of a divided by b."""
    if b == 0:
        raise ValueError("Cannot take modulo by zero")
    return a % b
```

Commit and push the branch:

Terminal

```
git add calculator.py
git commit -m "Add modulo operation (Closes #1)"
git push -u origin feature/modulo-operation
```

7.3 Opening a Pull Request

A Pull Request (PR) proposes merging your branch into main, and opens it up for review and discussion.

1. After pushing, GitHub shows a banner: **Compare & pull request**. Click it. (Or go to the **Pull requests** tab → **New pull request**.)
2. Set **base:** `main` and **compare:** `feature/modulo-operation`. This means "merge my feature branch into main."
3. Give it a clear title: `Add modulo operation to calculator`.
4. Write a description of what changed and why. Reference the Issue:

Pull Request description

```
## What this does
Adds a `modulo(a, b)` function returning the remainder of a / b.

## Why
Requested in #1.

## Testing
- modulo(10, 3) returns 1
- modulo(5, 0) raises ValueError

Closes #1
```

5. On the right, add a **Reviewer** (a classmate or your instructor). Click **Create pull request**.

NOTE

The PR page now shows three useful tabs: **Conversation** (discussion), **Commits** (what changed), and **Files changed** (the actual diff). This is where review happens.

7.4 Reviewing code on GitHub

Now play the role of the reviewer. Code review is how teams catch bugs and share knowledge before code reaches main.

1. Open the PR and click the **Files changed** tab. You'll see every added line in green, removed lines in red.
2. Hover over any line and click the blue **+** to leave a comment on that exact line. For example, suggest a test or point out an edge case.
3. You can propose an exact code change with a **suggestion**: in the comment box, click the **± (Insert suggestion)** button and edit the code. The author can accept it with one click.
4. When done reviewing, click **Review changes** (top-right of Files changed) and choose one:
 - **Comment** — general feedback, no explicit approval.
 - **Approve** — the change looks good to merge.
 - **Request changes** — something must be fixed before merging.
5. Add a summary comment and click **Submit review**.

 TIP

If the reviewer requests changes, the author simply commits more work to the **same branch** and pushes. The PR updates automatically — no need to open a new PR.

If changes were requested, here's how the author responds:

Terminal

```
# You're still on feature/modulo-operation
# ...make the requested edits to calculator.py...

git add calculator.py
git commit -m "Add input validation per review feedback"
git push

# The open PR picks up the new commit automatically
```

7.5 Merging the Pull Request

Once the PR is approved, merge it into main — all from the GitHub web page.

1. On the PR's **Conversation** tab, scroll to the merge box at the bottom.
2. Click the **Merge pull request** dropdown and pick a strategy:
 - **Create a merge commit** — keeps every commit plus a merge commit. Full history.
 - **Squash and merge** — combines all the branch's commits into one clean commit. Most popular for features.
 - **Rebase and merge** — replays commits onto main with no merge commit. Linear history.
3. Click **Confirm merge**. Because the PR said **Closes #1**, Issue #1 closes automatically.
4. Click **Delete branch** — the feature branch has served its purpose.

7.6 Sync your local machine

The merge happened on GitHub. Pull it down so your local main matches:

Terminal

```
git checkout main
git pull

# Delete your local copy of the merged branch
git branch -d feature/modulo-operation
```

 NOTE

The full professional loop you just completed: Issue → branch → commit → push → Pull Request → review → merge → pull. This cycle repeats for every feature on every real-world team.

8. Doing Everything from Your IDE (VS Code)

You don't have to memorize commands — VS Code has Git built in, plus an extension that brings Issues and Pull Requests right into the editor. Here's how every task above maps to the IDE.

8.1 One-time setup

1. Install **VS Code** and make sure Git is installed (run `git --version` in a terminal).
2. Open the **Extensions** panel (square icon in the sidebar) and install **GitHub Pull Requests and Issues** (by GitHub).
3. Click **Sign in** when prompted to connect your GitHub account — this lets VS Code manage Issues and PRs for you.

8.2 The Source Control panel

Click the **Source Control** icon in the left sidebar (it looks like a branching road, shortcut `Ctrl+Shift+G`). This single panel replaces most Git commands:

Action in VS Code	What it replaces
Open a folder	File → Open Folder, then Source Control → Initialize Repository (= git init).
Stage a file	Click the + next to the file (= git add).
Commit	Type a message in the box and click ✓ Commit (= git commit).
Push / Pull	Use the ... menu or the sync arrows in the status bar (= git push / git pull).
See changes	Click any file to open a side-by-side diff (= git diff).

8.3 Branching from the IDE

Look at the **bottom-left status bar** — it shows your current branch name. Click it to:

- See all branches and switch between them.
- Create a new branch (**Create new branch...**) — the equivalent of `git checkout -b`.

The **sync icon** (circular arrows) next to the branch name pushes and pulls in one click. A number badge tells you how many commits you're ahead or behind.

8.4 Resolving conflicts in the IDE

When a merge conflicts, VS Code highlights the conflicting regions and shows clickable buttons directly above them:

- **Accept Current Change** — keep your branch's version.
- **Accept Incoming Change** — keep the other branch's version.
- **Accept Both Changes** — keep both, stacked.

- Or open the **Merge Editor** for a three-pane view that makes complex conflicts easy.

After resolving, stage the file and commit — exactly as in Section 6, but with buttons instead of editing markers.

8.5 Issues and Pull Requests inside VS Code

This is where the GitHub extension shines — you never open a browser.

Working with Issues

- A **GitHub** icon appears in the sidebar with an **Issues** section listing your repo's open issues.
- Click an issue to read it. Click **Start working on issue** and VS Code **automatically creates a branch** for it.
- Create a new issue with the **+** in the Issues section.

Working with Pull Requests

1. After committing on your branch, push, then open the **Pull Requests** section in the GitHub sidebar.
2. Click **Create Pull Request**. Fill in the base branch, title, and description right inside the editor.
3. Reviewers can open the PR in VS Code, browse **Files Changed**, and add review comments line-by-line — without leaving the IDE.
4. When approved, click **Merge** from the PR view in VS Code.

TIP

The IDE workflow and the web workflow are interchangeable — they act on the same GitHub repo. Use the website to learn what's happening, then switch to the IDE for speed once you're comfortable.

8.6 IDE shortcuts cheat sheet

Shortcut / Action	Result
Ctrl+Shift+G	Open the Source Control panel.
Stage / Unstage	Hover a file in Source Control and click + or - .
Commit	Type message, press Ctrl+Enter.
Sync (push+pull)	Click the sync arrows in the bottom status bar.
Switch branch	Click the branch name in the bottom-left.
View file history	Right-click a file → Git: View File History.

9. Bonus: Other Essential Commands

These round out your toolkit. You won't need all of them today, but they come up constantly in real work.

9.1 Undoing things

Terminal

```
# Discard edits to a file (restore last committed version)
git restore hello.py

# Unstage a file (keep the edits, just remove from staging)
git restore --staged hello.py

# Undo the last commit but KEEP the changes in your working dir
git reset --soft HEAD~1

# Undo the last commit and DISCARD the changes (careful!)
git reset --hard HEAD~1

# Safely reverse a commit by making a new "opposite" commit
git revert <commit-hash>
```

⚠ WATCH OUT

`git reset --hard` permanently deletes uncommitted work. Prefer `git revert` on anything you've already pushed — it undoes safely without rewriting shared history.

9.2 Stashing — set work aside temporarily

Need to switch branches but aren't ready to commit? Stash your changes:

Terminal

```
# Shelve all uncommitted changes
git stash

# See your list of stashes
git stash list

# Bring the most recent stash back and remove it from the list
git stash pop
```

9.3 Tagging releases

Terminal

```
# Create an annotated tag for a release
git tag -a v1.0 -m "First stable release of github_lab"

# Push tags to GitHub
git push origin --tags
```

9.4 Full bonus reference

Command	What it does
<code>git restore <file></code>	Discard working-directory changes to a file.
<code>git restore --staged <file></code>	Remove a file from staging (keep edits).
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged.
<code>git reset --hard HEAD~1</code>	Undo last commit and delete the changes.
<code>git revert <hash></code>	Create a new commit that reverses an old one.
<code>git stash</code>	Temporarily shelve uncommitted changes.
<code>git stash pop</code>	Restore the most recent stash.
<code>git tag -a v1.0 -m "..."</code>	Create an annotated release tag.
<code>git fetch</code>	Download remote changes without merging.
<code>git rm <file></code>	Delete a file and stage the deletion.
<code>git mv <old> <new></code>	Rename/move a file and stage it.

10. Lab Tasks & Submission Checklist

Complete the following and show your repository to your instructor. Each task maps to a section of this manual.

Part A — Local Git

- A1.** Create the blank `github_lab` repo on GitHub and configure Git locally.
- A2.** Add `hello.py` and `.gitignore`, commit, and push.
- A3.** Add `calculator.py` in a second commit and push.
- A4.** Run `git log --oneline --graph --all` and screenshot the output.

Part B — Branching & Conflicts

- B1.** Create `feature/power-function`, add the `power` function, and merge it into main.
- B2.** Deliberately create and resolve a merge conflict in `hello.py` (Section 6).

Part C — GitHub Collaboration

- C1.** Open an Issue requesting the `modulo` function.
- C2.** Create `feature/modulo-operation`, implement it, and open a Pull Request that says `Closes #1`.
- C3.** Have a classmate review the PR (leave at least one line comment and Approve).
- C4.** Merge the PR, confirm the Issue auto-closed, and pull main locally.

Part D — From the IDE

D1. Repeat one full feature cycle (branch → commit → push → PR → merge) entirely inside VS Code.

 TIP

Submission: Submit the URL of your `github_lab` repository, plus screenshots of (1) your commit graph, (2) the resolved conflict commit, and (3) your merged Pull Request showing the linked Issue closed.

End of Lab Manual

Keep this manual open as a reference for all future labs.